DATA ITEM DESCRIPTION

Title: NUCLEAR SAFETY SOFTWARE VERIFICATION PLAN

Number: DI-NUOR-82402
Approval Date: 20230203
AMSC Number: F10377
Limitation: N/A
DTIC Applicable: No
GIDEP Applicable: No
Preparing Activity: 37 (AFSEC/SEWN)
Project Number: NUOR-2022-002
Applicable Forms: N/A

Use/Relationship: The Nuclear Safety Software Verification Plan (NSSVP) describes the purpose, methods, and procedures for the verification effort performed by the Independent Verification Organization (IVO) as part of Nuclear Safety Design Certification (NSDC). Headquarters Air Force Safety Center Nuclear Weapons Safety Branch (AFSEC/SEWN) provides NSDC to system program offices for specific nuclear weapon systems and nuclear weapon system modifications.

The NSSVP is described in Chapter 4 of Department of the Air Force Manual (DAFMAN) 91-119, Safety Design and Evaluation Criteria for Nuclear Weapon Systems Software. DAFMAN 91-119 implements Department of the Air Force Instruction (DAFI) 91-101, Air Force Nuclear Weapons Surety Program. DAFI 91-101 explains the process for NSDC. Both DAFMAN 91-119 and DAFI 91-101 are available without restriction on the Air Force E-Publishing website, https://www.e-publishing.af.mil/.

a. This DID is applicable to the acquisition of software for nuclear weapon systems that must be nuclear safety design certified per DAFI 91-101. Per Chapter 2 of DAFMAN 91-119, software is "…application software that executes sequentially and reconfigurable logic implementations that use a hardware description language for development."

b. Information required to implement this DID is defined in DAFMAN 91-119 and DAFI 91-101. This information includes procedures and plans to assure independence between the Development Organization (DO) and the IVO, compensatory measures to account for concerns in that independence, methods and procedures used to perform the verification effort, Nuclear Safety Objectives (NSO) and Nuclear Safety Requirements (NSR), and traceability between NSOs, NSRs, DAFMAN 91-119 requirements, and verification procedures, tools, and methodologies.

c. This DID contains the content and intended use information for the data deliverable resulting from the work task described in the solicitation and defined in Chapter 4 of DAFMAN 91-119.

d. The U.S. Navy is excluded from requirements set forth in this DID.

Source: https://assist.dla.mil -- Downloaded: 2026-10-26T11:03Z
Check the source to verify that this is the current version before use.

Requirements:

1. **Reference documents**. The applicable issue of the documents cited herein, including their approval dates and dates of any applicable amendments, notices, and revisions, shall be as specified in the contract.

2. **Format**. The NSSVP shall be in contractor's format. If the contractor chooses to provide the NSSVP as a website, the contractor shall provide a static snapshot of the complete website that can be viewed locally.

   2.1 The NSSVP is not limited in length but is limited in scope per the Content portion of this DID. If the IVO does not provide the completed NSSVP to AFSEC/SEWN before the IVO performs the finalized verification activities, AFSEC/SEWN may not accept the results of the verification activities.

   2.2 The items described within the Content portion of this DID do not need to be in the same document. The contractor may separate the NSSVP into multiple cohesive documents for ease of delivery, particularly the verification procedures described in 3.9. The contractor may make the sections available to AFSEC/SEWN for review as the contractor completes the sections.

   2.3 Table 4.1 of DAFMAN 91-119 provides additional information on expected content for the NSSVP, but the IVO should not treat information in Table 4.1 as a requirement.

   2.4 The NSSVP shall meet all requirements for NSSVPs found in DAFMAN 91-119. This includes restrictions found in DAFMAN 91-119 that the IVO cannot share the NSSVP with the DO without permission of AFSEC/SEWN.

3. **Content**. The NSSVP shall only contain the information relevant to NSDC of the software as described in DAFMAN 91-119 and DAFI 91-101. In particular, the NSSVP shall not contain information for the development of Weapon System Safety Rules.

   3.1 If the NSSVP is separated into multiple documents to ease delivery as described in 2.2 of this DID, the NSSVP shall include an overview of the internal layout of the document. If the NSSVP is a static snapshot of a website, the overview can be a sitemap.

   3.2 The NSSVP shall define the nuclear weapon system that the IVO contractor is verifying as listed in 3.2.1 through 3.2.3.

      3.2.1 Short description of the nuclear weapon system.

      3.2.2 Breakdown of the nuclear weapon system into components, focusing on the software components. The IVO may include the Critical Signal Analysis as defined in Chapter 3 of DAFMAN 91-119 if that inclusion simplifies the definition of software

Source: https://assist.dla.mil -- Downloaded: 2026-10-26T11:03Z
Check the source to verify that this is the current version before use.

components.

3.2.3 Type of verification for each software component, either Nuclear Safety Cross-Check Analysis (NSCCA) or Nuclear Safety Analysis and Technical Evaluation (NSATE).

3.3 The NSSVP shall define the IVO, including general organization of the team and the parent organization(s) of the IVO, including all contractors that are performing verification efforts specifically for the nuclear weapon system. The NSSVP shall duplicate 3.3 and 3.4 of this DID for each organization included in the IVO. The IVO includes any organization that performs work specifically for NSDC software verification of the system described in 3.2 of this DID.

3.4 The NSSVP shall define the DO and explain connections and potential independence issues between the DO and the IVO by answering the questions listed in 3.4.1 through 3.4.3 of this DID.

3.4.1 Managerial Independence:

3.4.1.1 Are the DO and IVO separate organizations? If not, at what level of management is the management chain common?

3.4.1.2 Are there any conflicts of interest between the management team of the DO and the management team of the IVO?

3.4.1.3 If there are Managerial Independence concerns, what compensatory measures will the IVO take to ensure full Managerial Independence?

3.4.2 Financial Independence:

3.4.2.1 Are there any financial incentives for the IVO to be less thorough in their verification process? Does the IVO perform separate work for the DO, as an example?

3.4.2.2 If there are Financial Independence concerns, what compensatory measures will the IVO take to ensure full Financial Independence?

3.4.3 Technical Independence:

3.4.3.1 Are there any situations where the IVO and DO must share resources, including facilities, lab equipment, or personnel such as lab personnel?

Source: https://assist.dla.mil -- Downloaded: 2026-10-26T11:03Z
Check the source to verify that this is the current version before use.

3.4.3.2 What compensatory measures are in place to ensure Technical Independence when the IVO and the DO share resources? What locations and situations do the compensatory measures cover and are there additional situations that may arise?

3.4.3.3 What procedures are in place to ensure that the IVO does not share verification data with the DO? What data do these procedures cover?

3.4.3.4 Are there any situations protecting the verification data (processes, procedures, methods, tools, configurations, etc.) from the DO will be difficult for the IVO due to program design or restrictions?

3.4.3.5 What compensatory measures are in place to ensure Technical Independence when situations arise that may provide verification data to the DO?

3.4.3.6 How will the IVO independently identify, acquire, and develop as necessary the verification suite and development suite components? What components will the IVO need to acquire from the DO? How will the IVO ensure that the software tools acquired from the DO will not affect the outcomes of verification processes?

3.4.3.7 What measures are in place to ensure that all members of the IVO sign a non-disclosure agreement stating that the member will not provide any verification data outside of the IVO?

3.4.3.8 What general procedures are in place to ensure that the IVO maintains independence from the DO?

3.4.3.9 When AFSEC/SEWN requires NSCCA, what procedures are in place to ensure the NSCCA team independently develops their own verification requirements and procedures that focus on identifying sabotage or unintended actions that could violate nuclear safety?

3.5 When AFSEC/SEWN requires NSCCA, the NSSVP shall describe configuration management systems and security and control measures as listed in 3.5.1 and 3.5.2.

3.5.1 Description of the configuration management systems the IVO will use for maintaining software source code, software products, verification tools, and verification procedures, and how the configuration management systems are secure and non-repudiable.

3.5.2 Description of the security and control measures in place in the IVO to prevent sabotage of the NSCCA process. The description may reference the contractor

Source: https://assist.dla.mil -- Downloaded: 2026-10-26T11:03Z
Check the source to verify that this is the current version before use.

standard security processes as well as additional measures that the IVO is taking.

3.6 The NSSVP shall describe the development and verification software tools the IVO plans to use during the verification effort, both created and acquired by the IVO, including the information listed in 3.6.1 through 3.6.4. (see Figure 1).

3.6.1 Name of software tool.

3.6.2 Software tool creator or manufacturer.

3.6.3 General description of how the IVO uses the software tool.

3.6.4 If the software tool is provided by the DO, reference to the information provided in 3.4.3.6 of this DID addressing software tools provided by the DO.

3.7 The NSSVP shall describe the NSOs developed by the IVO and how the NSOs trace to DAFMAN 91-119. The definition of NSO is in Chapter 4 of DAFMAN 91-119. (see Figure 2).

3.8 The NSSVP shall describe the NSRs developed by the IVO and how the NSRs trace to the NSOs. The definition of NSR is in Chapter 4 of DAFMAN 91-119. (see Figure 3).

3.9 The NSSVP shall include a summary of all the verification procedures that includes an overview of the procedure and to which NSR the procedure traces. For NSSVPs that include fewer than 10 procedures, this paragraph may be tailored out. (see Figure 4).

3.10 The NSSVP shall include the full set of verification procedures within the greater document or as a separate attachment. Each procedure shall reference the NSR to which the procedure traces as well as verification type(s) – Test, Demonstration, Inspection, Analysis.

3.11 The NSSVP shall include a full traceability analysis flowing from the DAFMAN 91-119 requirements → NSOs → NSRs → Verification procedures and the reverse trace flowing from the Verification procedures → NSRs → NSOs → DAFMAN 91-119 requirements. The traceability analysis shall be easily navigable to ease the search for untraced objects within the analysis. The analysis may be a static snapshot of a website with links showing connections from one level to the next. (see Figure 5 and Figure 6).

Source: https://assist.dla.mil -- Downloaded: 2026-10-26T11:03Z
Check the source to verify that this is the current version before use.

Tool Name	Creator/ Manufacturer	General Description of Use	Created by DO?	Reason for Confidence in DO tools
XYZ Static Analysis Tool	XYZ, Inc.	Analysis tool to identify coding standard violations, including analysis of MISRA C and C++ standards.	No	N/A
Company A Build Script	Company A	Script used to build DO software into object code for both test environment and target hardware.	Yes	SW Team reviewed script and associated libraries, analyzed output of build for both test and target, executed cybersecurity tools on output to look for indicators of unexpected code insertion.

FIGURE 1. Sample software tool entry.

DAFMAN 91-119 Requirement	NSO
8.15.1. The software shall not contain unused code.	1.1.1 The verification process shall prove that the software does not contain code that is untestable except for justifiable circumstances.
8.15.2. The software shall not contain global variables.	1.1.2 The verification process shall prove that the software does not contain global variables.
8.15.5. The software shall not execute memory allocation functions after application initialization.	1.1.3 The verification process shall prove that there is no portion of the software that executes “new” or “malloc” functionality once the initAllTasks function is complete.

FIGURE 2. Sample NSO entry.

NSO	NSR
1.1.1 The verification process shall prove that the software does not contain code that is untestable except for justifiable circumstances.	1.1.1.A – The verifier shall execute XYZ Static Analysis Tool to identify obvious non-executable sections of code. If the tool identifies any sections of code as non-executable, the verifier shall analyze the sections to determine if the identification is a false positive. The verifier shall request another verifier perform the same analysis on identified sections. 1.1.1.B – The verifier shall execute dynamic testing using ABC code coverage tool to identify non-executable sections of code. If the tool identifies any sections of code as non-executable, the verifier shall analyze the sections to determine if the identification

Source: https://assist.dla.mil -- Downloaded: 2026-10-26T11:03Z
Check the source to verify that this is the current version before use.

NSO	NSR
	is a false positive. The verifier shall request another verifier perform the same analysis on identified sections. 1.1.1.C – If the verifiers identify any sections of code as non-executable using the above steps and the verifiers have shown that the identifications are not a false positive, the verifiers shall analyze the identified sections to determine if the software section falls under a justifiable exemption (nature of the code, error handling in the code or low-level source code the software developer designed for very specific scenarios.) 1.1.1.D – If the verifier cannot instrument a section of code for dynamic analysis, the verifier shall analyze the section of code to determine if there are un-executable sections. If the analysis identifies any sections of code as non-executable, the verifier shall request another verifier perform the same analysis on identified sections. The verifiers shall then meet NSR 1.1.1.C.
1.1.2 The verification process shall prove that the software does not contain global variables.	1.1.2.A – The verifier shall execute XYZ Static Analysis Tool to identify global variables or global const declarations, including any extern declarations, using the following definition of global: An object that allows direct data manipulation across different segments or functional sections without using data encapsulation methods. If the tool identifies any variable, functions, or declarations as global, the verifier shall analyze the sections to determine if the identification is a false positive. The verifier shall request another verifier perform the same analysis on identified sections.
1.1.3 The verification process shall prove that there is no portion of the software that executes “new” or “malloc” functionality once the initAllTasks function is complete.	1.1.3.A The verifier shall use the built-in operating system tool to identify any memory allocation that occurs after the initAllTasks function is complete. If the tool identifies any unacceptable memory allocations, the verifier shall analyze the software to determine if the identification is a false positive. The verifier shall request another verifier perform the same analysis on identified software.

FIGURE 3. Sample NSR entry.

NSR	Verification Procedure Summary
1.1.1.A - The verifier shall execute XYZ Static Analysis Tool to identify obvious non-executable sections of code. If the tool identifies any sections of code as non-executable, the verifier shall analyze the sections to determine if the identification is a false positive. The verifier shall request another verifier perform the same analysis on identified sections.	Procedure 13 – Steps to identify obvious non-executable code and reject false positives through analysis.

Source: https://assist.dla.mil -- Downloaded: 2026-10-26T11:03Z
Check the source to verify that this is the current version before use.

NSR	Verification Procedure Summary
1.1.2.A – The verifier shall execute XYZ Static Analysis Tool to identify global variables or global const declarations, including any extern declarations, using the following definition of global: An object that allows direct data manipulation across different segments or functional sections without using data encapsulation methods. If the tool identifies any variable, functions, or declarations as global, the verifier shall analyze the sections to determine if the identification is a false positive. The verifier shall request another verifier perform the same analysis on identified sections.	Procedure 42 – Steps to identify global variables, consts, and externs, and reject false positives through analysis.
1.1.3.A The verifier shall use the built-in operating system tool to identify any memory allocation that occurs after the initAllTasks function is complete. If the tool identifies any unacceptable memory allocations, the verifier shall analyze the software to determine if the identification is a false positive. The verifier shall request another verifier perform the same analysis on identified software.	Procedure 143 – Steps to identify memory allocations after initialization is complete and reject false positives through analysis.

FIGURE 4. Sample verification procedure summary entry.

DAFMAN 91-119 Requirement	NSO	NSR	Verification Procedure
8.15.1. The software shall not contain unused code.	1.1.1 The verification process shall prove that the software does not contain code that is untestable except for justifiable circumstances.	1.1.1.A – The verifier shall execute XYZ Static Analysis Tool to identify obvious non-executable sections of code. If the tool identifies any sections of code as non-executable, the verifier shall analyze the sections to determine if the identification is a false positive. The verifier shall request another verifier perform the same analysis on identified sections.	Procedure 13
8.15.2. The software shall not contain global variables.	1.1.2 The verification process shall prove that the software does not contain global variables.	1.1.2.A – The verifier shall execute XYZ Static Analysis Tool to identify global variables or global const declarations, including any extern declarations, using the following definition of global: An object that allows direct	Procedure 42

Source: https://assist.dla.mil -- Downloaded: 2026-10-26T11:03Z
Check the source to verify that this is the current version before use.

DAFMAN 91-119 Requirement	NSO	NSR	Verification Procedure
		data manipulation across different segments or functional sections without using data encapsulation methods. If the tool identifies any variable, functions, or declarations as global, the verifier shall analyze the sections to determine if the identification is a false positive. The verifier shall request another verifier perform the same analysis on identified sections.	
8.15.5. The software shall not execute memory allocation functions after application initialization.	1.1.3 The verification process shall prove that there is no portion of the software that executes “new” or “malloc” functionality once the initAllTasks function is complete.	1.1.3.A The verifier shall use the built-in operating system tool to identify any memory allocation that occurs after the initAllTasks function is complete. If the tool identifies any unacceptable memory allocations are identified the verifier shall analyze the software to determine if the identification is a false positive. The verifier shall request another verifier perform the same analysis on identified software.	Procedure 143

FIGURE 5. Sample verification forward trace entry.

Verification Procedure	NSR	NSO	DAFMAN 91-119 Requirement
Procedure 13	1.1.1.A – The verifier shall execute XYZ Static Analysis Tool to identify obvious non-executable sections of code. If the tool identifies any sections of code as non-executable, the verifier shall analyze the sections to determine if the identification is a false	1.1.1 The verification process shall prove that the software does not contain code that is untestable except for justifiable circumstances.	8.15.1. The software shall not contain unused code.

Source: https://assist.dla.mil -- Downloaded: 2026-10-26T11:03Z
Check the source to verify that this is the current version before use.

Verification Procedure	NSR	NSO	DAFMAN 91-119 Requirement
	positive. The verifier shall request another verifier perform the same analysis on identified sections.		
Procedure 42	1.1.2.A – The verifier shall execute XYZ Static Analysis Tool to identify global variables or global const declarations, including any extern declarations, using the following definition of global: An object that allows direct data manipulation across different segments or functional sections without using data encapsulation methods. If the tool identifies any variable, functions, or declarations as global, the verifier shall analyze the sections to determine if the identification is a false positive. The verifier shall request another verifier perform the same analysis on identified sections.	1.1.2 The verification process shall prove that the software does not contain global variables.	8.15.2. The software shall not contain global variables.
Procedure 143	1.1.3.A The verifier shall use the built-in operating system tool to identify any memory allocation that occurs after the initAllTasks function is complete. If the tool identifies any unacceptable memory allocations, the verifier shall analyze the software to determine if the identification is a false positive. The verifier shall request another verifier perform the same analysis on identified software.	1.1.3 The verification process shall prove that there is no portion of the software that executes “new” or “malloc” functionality once the initAllTasks function is complete.	8.15.5. The software shall not execute memory allocation functions after application initialization.

FIGURE 6. Sample verification reverse trace entry.

End of DI-NUOR-82402.

Source: https://assist.dla.mil -- Downloaded: 2026-10-26T11:03Z
Check the source to verify that this is the current version before use.